BONAFIDE CERTIFICATE

This is to certify that this project work titled **“ Posture Assessment using Machine Learning”** is a Bonafide record of work carried out by Mr. Shiva Shankar Peddi, Mr. Pabba Sai Sathwik, Mr. Srikar Chidara, Mr. Sri Nandan Gondi, and submitted to Centre for Training and Learning(CTL), National Institute of Technology, Warangal under my supervision from **“20 May 2024** to **“20 June 2024”**

Supervisor

Prof. T. Kishore Kumar

Professor, Department of ECE

NIT Warangal

DECLARATION

We hereby declare that the work presented in this report titled, “Intelligent Posture assessment using machine learning” is the result of our research and effort. We have cited all sources of information that have been used in the preparation of this report, ensuring that due credit is given to the authors and researchers whose work has contributed to this study. We affirm that this report has not been submitted previously, either in part or in full, for any academic or professional qualification or degree at any other institution.

Furthermore, we have taken all necessary steps to ensure the integrity and authenticity of this report. Any assistance or guidance received during this project has been appropriately acknowledged. We take full responsibility for any errors or omissions contained within this report and am prepared to answer any questions or provide additional information regarding its contents.

Place: Warangal

Date: 19 June 2024

TABLE OF CONTENTS

ACKNOWLEDGEMENT:

We would like to express our sincere gratitude to all those who have supported and guided us throughout the completion of this report. First and foremost, we extend our deepest appreciation to Dr. T. Kishore Kumar Sir, Prof, Head of CTL, NIT Warangal, whose invaluable guidance, constructive feedback, and continuous encouragement were instrumental in shaping the direction and quality of this work. Their expertise and insight have been crucial in navigating the challenges encountered during this research.

We are also grateful to our peers and colleagues, whose collaborative spirit and thoughtful discussions have enriched this project. Special thanks to Dr. K. Sunil Kumar, PhD Scholar, for their technical assistance and to Manoj Reddy Sir, and Suneetha Sivakrishna Ma'am for their help in data collection and analysis. Lastly, we would like to thank my family and friends for their unwavering support and understanding, which was a source of motivation and inspiration throughout this journey. This report would not have been possible without the collective effort and contribution of all these individuals.

We wish a deep sense of gratitude and heartfelt thanks to management for providing excellent lab facilities and tools. Finally, we thank our seniors whose guidance helped us in this regard. Completion of this project and thesis would not have been possible without the help of many people, to whom we are very thankful.

ABSTRACT:

This project focuses on the development of a posture assessment tool utilizing machine learning techniques to analyse and evaluate human posture in real-time. Leveraging Media Pipe, a powerful framework for building multimodal perception solutions, we can accurately track and measure the angles of various body parts, including the hands, arms, legs, and neck. Our system provides immediate feedback on the user's posture, identifying any deviations from optimal alignment and offering corrective suggestions. The primary objective of this project is to promote better posture habits and prevent potential musculoskeletal issues by providing users with precise and actionable insights.

Unlike traditional posture assessment methods, our approach does not require extensive training of custom models. Instead, we harness the pre-trained capabilities of Mediapipe to deliver high-accuracy results with minimal computational overhead. This allows for efficient and scalable posture analysis, making it accessible for a wide range of applications, from personal health monitoring to ergonomic assessments in workplaces. The system's ability to provide real-time feedback makes it particularly valuable for users seeking immediate corrections and long-term posture improvement.

INTRODUCTION:

Maintaining proper posture is essential for overall health, as poor posture can lead to a range of musculoskeletal issues, including chronic back pain, neck strain, and decreased mobility. In an era where prolonged sitting is prevalent due to increased screen time and remote work, the importance of effective posture monitoring has never been more critical. Studies indicate that up to 80% of adults experience back pain at some point in their lives, often due to poor posture. Moreover, the World Health Organization has identified musculoskeletal disorders as a leading cause of disability worldwide, underscoring the need for proactive posture management solutions.

Artificial Intelligence (AI) offers transformative potential in the realm of posture assessment by providing sophisticated tools for real-time monitoring and feedback. By leveraging machine learning algorithms, these systems can assess body alignment and detect deviations from optimal posture with high precision. This capability is particularly beneficial in identifying subtle posture issues that may not be apparent to the untrained eye, thus enabling early intervention and correction.

One of the key advantages of using AI in posture assessment is its ability to process and analyze large volumes of data quickly. With the integration of computer vision and pose estimation technologies, AI can continuously monitor an individual's posture during various activities, providing instant feedback and recommendations. This real-time analysis helps users make immediate adjustments, reducing the risk of long-term damage caused by poor posture. Additionally, AI systems can learn from user data over time, offering personalized insights and tailored recommendations to improve posture habits progressively.

Furthermore, the application of AI in posture assessment is not limited to personal health; it has significant implications for workplace ergonomics and athletic training. In office environments, AI can help design ergonomic workstations and promote healthier work habits, thereby enhancing employee well-being and productivity. In sports, AI-driven posture analysis can optimize performance and reduce injury risks by ensuring athletes maintain proper form during training and competition. As technology continues to advance, the integration of AI in posture assessment holds the promise of revolutionizing how we approach and maintain our physical health. In the realm of rehabilitation medicine, the integration of artificial intelligence (AI) has revolutionized the assessment and treatment of patients. Biomechanical analysis tools, leveraging

AI algorithms and advanced technology, offer unprecedented insights into human movement patterns, facilitating precise evaluations of gait, posture, and joint motion. This comprehensive understanding enables healthcare professionals to tailor rehabilitation techniques and interventions with unparalleled precision, optimizing outcomes and enhancing patient recovery.

Ultimately, the integration of AI in biomechanical analysis holds immense potential to transform rehabilitation medicine, improve patient outcomes, and enhance the overall quality of care.

PROJECT DESCRIPTION:

This project aims to address thE need by developing a posture assessment system that leverages machine learning techniques to analyse and evaluate human posture in real-time. By providing users with accurate feedback and actionable insights, our system seeks to promote better posture habits and prevent potential health problems associated with poor posture.

The core technology behind our posture assessment system is Mediapipe, a robust framework developed by Google that facilitates the creation of multimodal perception pipelines. Mediapipe offers pre-trained models for human pose estimation, which we utilize to track and measure the angles of various body parts, such as the hands, arms, legs, and neck. This approach eliminates the need for extensive custom model training, allowing us to achieve high-accuracy results with minimal computational resources. Our system processes real-time video input to assess the user's posture continuously, providing immediate feedback on any deviations from optimal alignment.

LITERATURE REVIEW:

Through the review of all the research on the role of AI in improving human body movement, and posture, the conclusion is that AI-powered tools are transforming biomechanical analysis in rehabilitation, utilizing machine learning and deep learning algorithms to assess movement, gait, posture, and joint motion. These tools have shown promise in various applications, aiding in the identification of abnormalities, tracking progress, and designing personalized interventions. Research has validated their accuracy against traditional methods, demonstrating comparable or superior performance.
Despite the progress, challenges remain, including data availability and quality, ethical considerations, and the need for interpretable AI models. Future directions include the development of more sophisticated algorithms, the integration of multimodal data, and the exploration of explainable AI techniques. AI-based biomechanical analysis tools have the potential to revolutionize rehabilitation practices by providing objective assessments, personalized interventions, and real-time feedback, ultimately improving the effectiveness and efficiency of care.

GAPS IDENTIFIED:

- Limited Real-Time Feedback
- Subjectivity in assessment
- Accessibility and Affordability
- Lack of Personalization
- Integration with daily activities
- Long-Term behaviour Change

By leveraging machine learning and computer vision, coupled with the accessibility and versatility of tools like Mediapipe, our project offers a comprehensive solution that addresses the limitations of existing methods. The real-time feedback capabilities of our system ensure immediate intervention and correction during activities, mitigating the risk of musculoskeletal issues caused by poor posture. Moreover, the objective and personalized nature of our AI-powered posture assessment tool enhances accuracy and effectiveness, bridging the gap of subjectivity and lack of customization in traditional approaches. Our project's affordability, user-friendliness, and seamless integration into daily activities make it accessible to a wide range of individuals. This can revolutionize how individuals monitor, assess, and improve their posture.

SOFTWARES AND LIBRARIES USED:

Media Pipe – Python module

OpenCV – Python module

NumPy – Python module

PROCEDURE:

- Collecting and organizing relevant training and testing data
- Installation of libraries
- Writing ML Code
- Training model with data
- Optimize and make model more accurate
    - **1.1.** Setting Up Media Pipe
    - **1.2.** Estimating Poses
    - **1.3.** Extracting joint Co-ordinates
    - **1.4.** Calculating angles between the joints
    - **1.5.** Build the necessary AI

DESIGN METHODOLOGY:

These are some basic steps that can be followed to achieve the objectives of this project.

- Problem definition and Scope
- Data Collection and Pre-Processing
- Model Selection and Training
- Integration with Mediapipe
- Real-time Feedback system
- User interface Development
- Testing and Validation
- Iterative Improvement

By following this design methodology, the posture assessment project can effectively leverage AI technologies, integrate with Media Pipe functionalities, and deliver a robust and user-friendly solution for real-time posture monitoring and correction.

BLOCK DIAGRAMS:

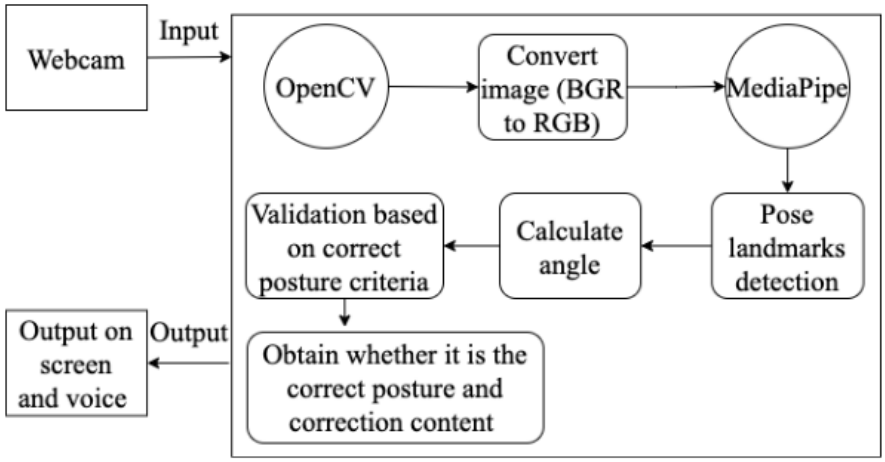


Fig. 1. Posture analysis flowchart

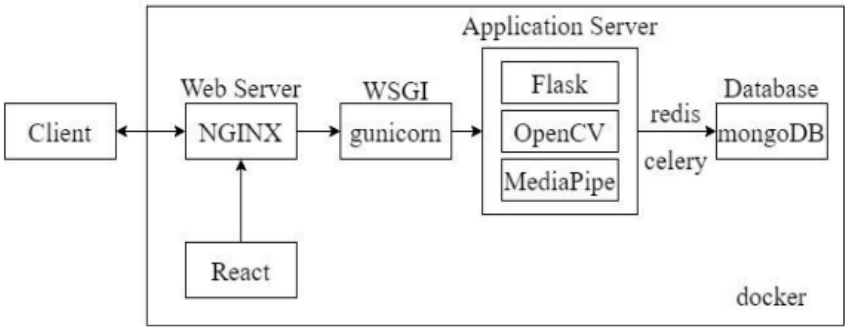
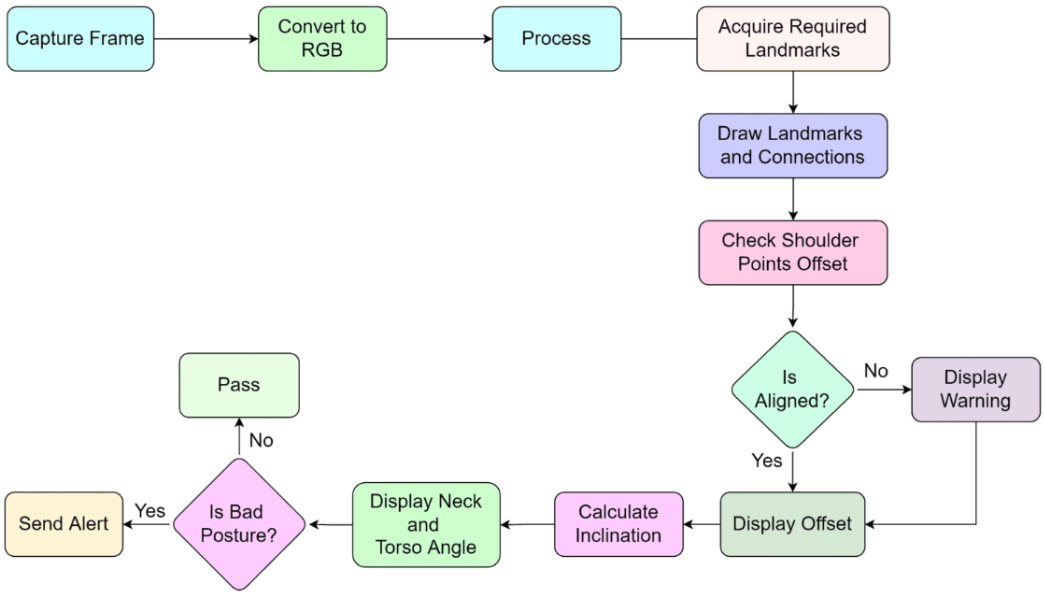
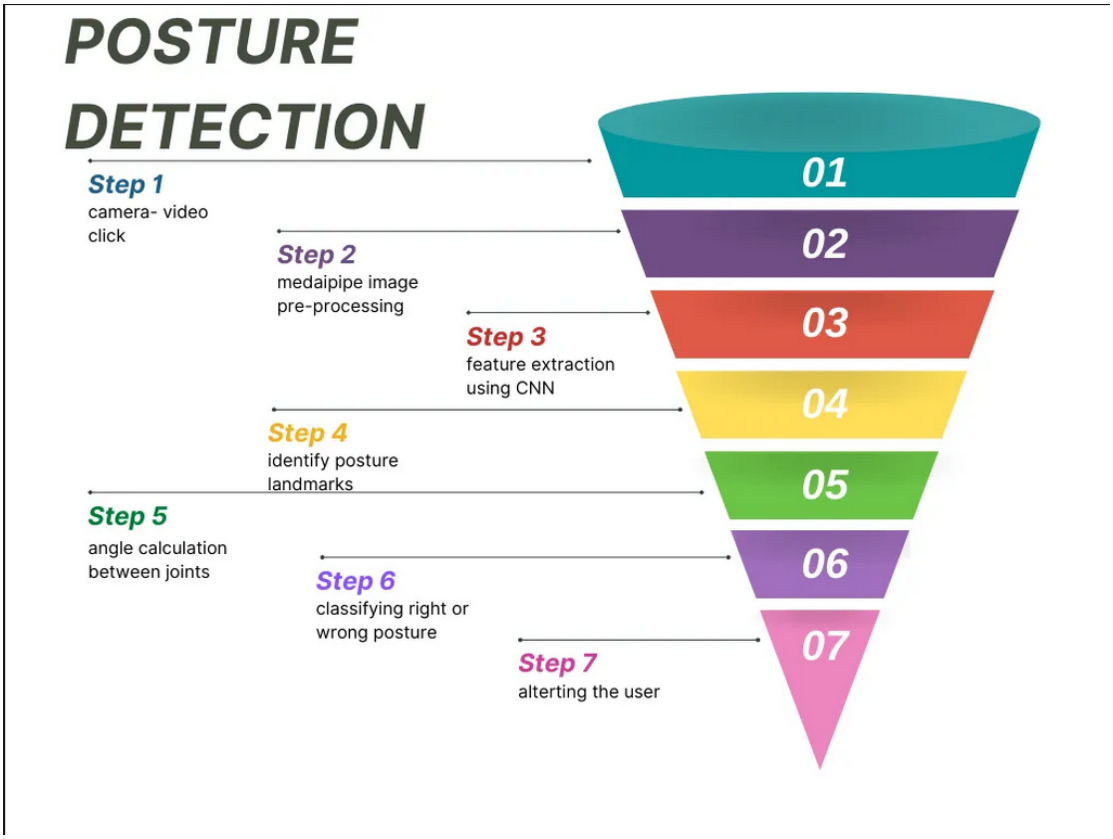


Fig. 2. System architecture




POSTURE
DETECTION
Step 1
camera- video
click
Step 2
medaipipe image
pre-processing
Step 3
feature extraction
using CNN
Step 4
identify posture
landmarks
Step 5
angle calculation
between joints
Step 6
classifying right or
wrong posture
Step 7
alterting the user
01
02
03
04
05
06
07


Reference
Standstill
Image
User
Image
Frame
Assessment
Text
Messages
User
Or
Trainee
Webcam

CODE:

```
pip install opencv-python mediapipe
```

```
import cv2
import mediapipe as mp
import numpy as np
```

```
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(static_image_mode=False, min_detection_confidence=0.5, min_tracking_confidence=0.5)
```

```
def calculate_angle(point1, point2, point3):
    a = np.array(point1)  # First
    b = np.array(point2)  # Mid
    c = np.array(point3)  # End

    radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(a[1] - b[1], a[0] - b[0])
    angle = np.abs(radians * 180.0 / np.pi)

    if angle > 180.0:
        angle = 360 - angle

    return angle
```

```
def is_side_view(landmarks):
    left_shoulder = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value]
    right_shoulder = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER.value]
    left_hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP.value]
    right_hip = landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value]

    # Simple check: Shoulders and hips should be vertically aligned
    shoulder_diff = abs(left_shoulder.y - right_shoulder.y)
    hip_diff = abs(left_hip.y - right_hip.y)

    if shoulder_diff < 0.1 and hip_diff < 0.1:
        return True
    return False
```

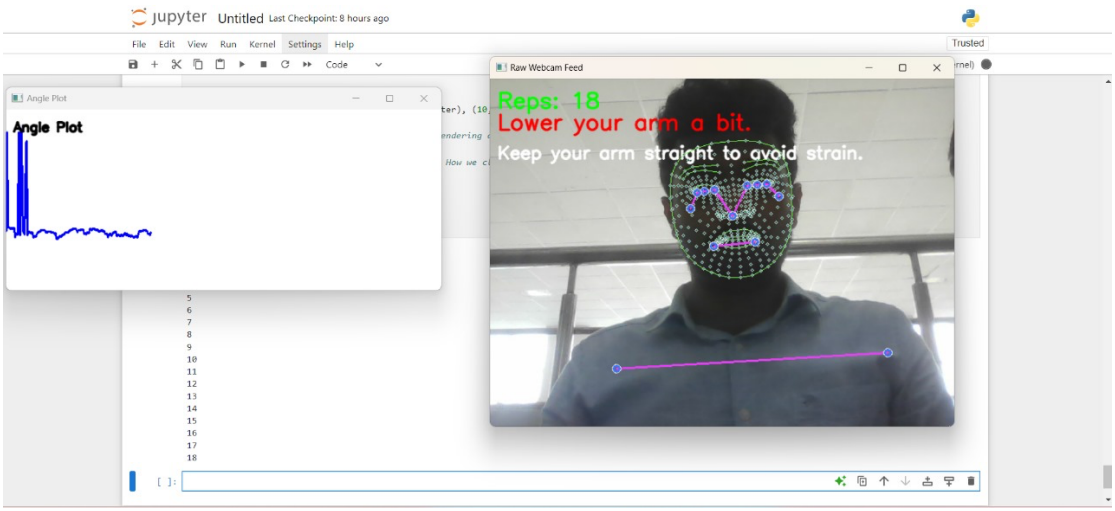
```
cap = cv2.VideoCapture(0)
threshold_angle = 30  # Example threshold

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = pose.process(frame_rgb)

    if results.pose_landmarks:
        landmarks = results.pose_landmarks.landmark

        # Check if the camera is aligned for the side view
        if is_side_view(landmarks):
            cv2.putText(frame, 'Side View: Aligned', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,
            cv2.LINE_AA)

            # Calculate neck and torso inclination
            neck_angle = calculate_angle(
                [landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
                 landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].y],
                [landmarks[mp_pose.PoseLandmark.NOSE.value].x, landmarks[mp_pose.PoseLandmark.NOSE.value].y],
                [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x, landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y]
            )

            torso_angle = calculate_angle(
                [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x,landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y],
                [landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
                landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].y],
                [landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x,
                 landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y]
            )

            cv2.putText(frame,
            f'Neck Angle: {int(neck_angle)}', (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
            cv2.putText(frame,
            f'Torso Angle: {int(torso_angle)}', (10, 110), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

            # Check if the person is bending below the threshold angle
            if neck_angle > threshold_angle:
                cv2.putText(frame, 'Neck Bend Detected', (10, 150),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
            if torso_angle > threshold_angle:
                cv2.putText(frame, 'Torso Bend Detected', (10, 190),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
        else:
            cv2.putText(frame, 'Side View: Not Aligned', (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)

    cv2.imshow('Frame', frame)

    if cv2.waitKey(10) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

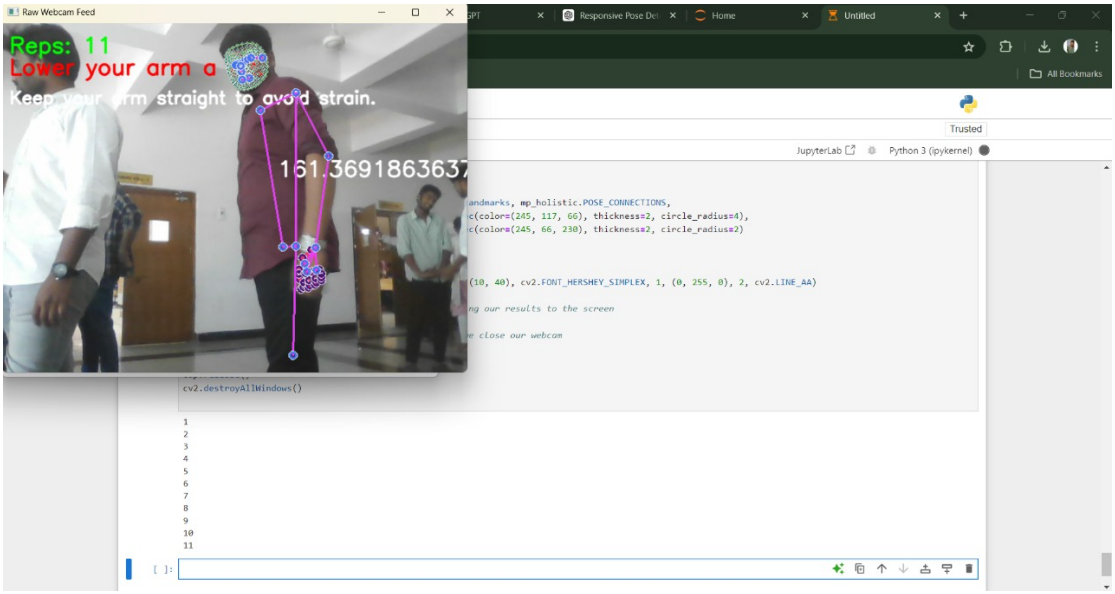
```python
import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe Pose
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(static_image_mode=False, min_detection_confidence=0.5, min_tracking_confidence=0.5)

# Function to calculate the angle between three points
def calculate_angle(point1, point2, point3):
    a = np.array(point1)  # First
    b = np.array(point2)  # Mid
    c = np.array(point3)  # End

    radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(a[1] - b[1], a[0] - b[0])
    angle = np.abs(radians * 180.0 / np.pi)

    if angle > 180.0:
        angle = 360 - angle

    return angle

# Function to check if the camera is properly aligned for side view
def is_side_view(landmarks):
    # Get coordinates of relevant landmarks
    left_shoulder = landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value]
    right_shoulder = landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER.value]
    left_hip = landmarks[mp_pose.PoseLandmark.LEFT_HIP.value]
    right_hip = landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value]

    # Calculate the absolute difference in y-coordinates
    shoulder_diff = abs(left_shoulder.y - right_shoulder.y)
    hip_diff = abs(left_hip.y - right_hip.y)

    # Check if the difference is within a small threshold
    # This means shoulders and hips are vertically aligned
    if shoulder_diff < 0.05 and hip_diff < 0.05:
        return True
    return False

# Main function to process video feed
def main():
    cap = cv2.VideoCapture(0)
    threshold_angle = 30  # Example threshold

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(frame_rgb)

        if results.pose_landmarks:
            landmarks = results.pose_landmarks.landmark

            # Check if the camera is aligned for the side view
            if is_side_view(landmarks):
                cv2.putText(frame, 'Side View: Aligned', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,
                cv2.LINE_AA)

                # Calculate neck and torso angles
                neck_angle = calculate_angle(
                    [landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
                     landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].y],
                    [landmarks[mp_pose.PoseLandmark.NOSE.value].x, landmarks[mp_pose.PoseLandmark.NOSE.value].y],
                    [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x,
                     landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y]
                )

                torso_angle = calculate_angle(
                    [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x,
                     landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y],
                    [landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
                     landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].y],
                    [landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x,
                     landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y]
                )

                cv2.putText(frame, f'Neck Angle: {int(neck_angle)}', (10, 70),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
                cv2.putText(frame, f'Torso Angle: {int(torso_angle)}', (10, 110),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

                # Check if the person is bending below the threshold angle
                if neck_angle > threshold_angle:
                    cv2.putText(frame, 'Neck Bend Detected', (10, 150),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
                if torso_angle > threshold_angle:
                    cv2.putText(frame, 'Torso Bend Detected', (10, 190),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
            else:
                cv2.putText(frame, 'Side View: Not Aligned', (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)

        cv2.imshow('Frame', frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

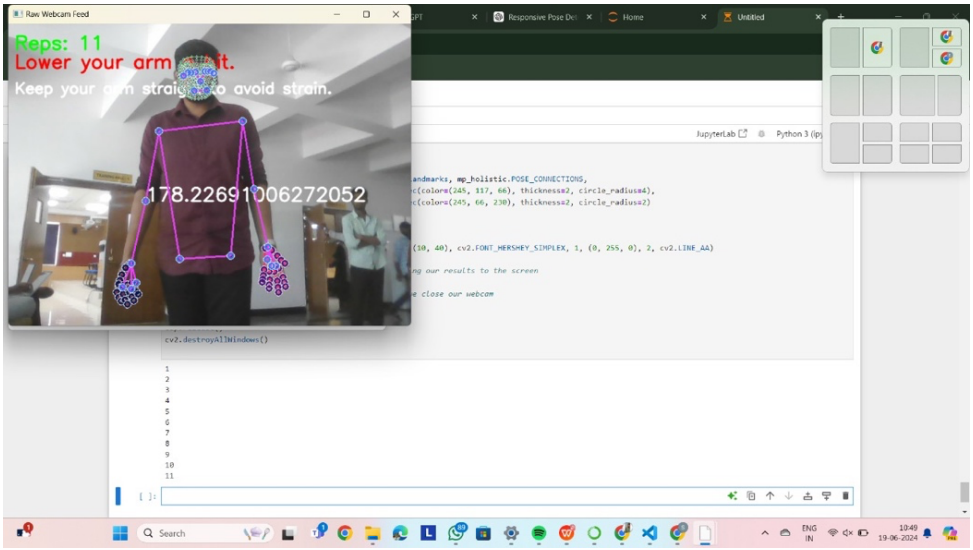
```python
import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe Holistic and Drawing utilities
mp_holistic = mp.solutions.holistic
mp_drawing = mp.solutions.drawing_utils
mp_face_mesh = mp.solutions.face_mesh

cap = cv2.VideoCapture(0)

# Function to calculate angle between three points
def calculate_angle(a, b, c):
    a = np.array(a)  # First point
    b = np.array(b)  # Mid point
    c = np.array(c)  # End point

    radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(a[1] - b[1], a[0] - b[0])
    angle = np.abs(radians * 180.0 / np.pi)

    if angle > 180.0:
        angle = 360 - angle

    return angle

# Variables to keep track of rep count
counter = 0
stage = None

# Initiate holistic model
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:
    while cap.isOpened():  # Check if webcam is open or not
        ret, frame = cap.read()  # Our image in the frame

        if not ret:
            break

        # Recolor Feed
        image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        image.setflags(write=0)  # Set writable to False

        # Make Detections
        results = holistic.process(image)  # Update results with new detections

        # Recolor image back to BGR for rendering
        image.setflags(write=1)  # Set writable to True
        image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)  # Recoloring back to RGB to BGR

        # Extract landmarks for the right arm
        if results.pose_landmarks:
            landmarks = results.pose_landmarks.landmark

            # Get coordinates
            shoulder = [landmarks[mp_holistic.PoseLandmark.RIGHT_SHOULDER.value].x,
                        landmarks[mp_holistic.PoseLandmark.RIGHT_SHOULDER.value].y]
            elbow = [landmarks[mp_holistic.PoseLandmark.RIGHT_ELBOW.value].x,
                     landmarks[mp_holistic.PoseLandmark.RIGHT_ELBOW.value].y]
            wrist = [landmarks[mp_holistic.PoseLandmark.RIGHT_WRIST.value].x,
                     landmarks[mp_holistic.PoseLandmark.RIGHT_WRIST.value].y]

            # Calculate angle
            angle = calculate_angle(shoulder, elbow, wrist)

            # Visualize angle
            cv2.putText(image, str(angle),
                        tuple(np.multiply(elbow, [640, 480]).astype(int)),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA
                        )

            # Curl counter logic
            if angle > 160:
                stage = "down"
            if angle < 30 and stage == 'down':
                stage = "up"
                counter += 1
                print(counter)

            # Calculate neck and torso inclination
            neck_angle = calculate_angle(
                [landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].y],
                [landmarks[mp_holistic.PoseLandmark.NOSE.value].x,
                 landmarks[mp_holistic.PoseLandmark.NOSE.value].y],
                [landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].y]
            )

            torso_angle = calculate_angle(
                [landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].y],
                [landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].y],
                [landmarks[mp_holistic.PoseLandmark.LEFT_ANKLE.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_ANKLE.value].y]
            )

            cv2.putText(image, f'Neck Angle: {int(neck_angle)}', (10, 70),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
            cv2.putText(image, f'Torso Angle: {int(torso_angle)}', (10, 110),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

            # Check if the person is bending below the threshold angle
            threshold_angle = 30  # Example threshold
            if neck_angle > threshold_angle:
                cv2.putText(image, 'Neck Bend Detected', (10, 150),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
            if torso_angle > threshold_angle:
                cv2.putText(image, 'Torso Bend Detected', (10, 190),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)

        # Draw face landmarks
        mp_drawing.draw_landmarks(image, results.face_landmarks, mp_face_mesh.FACEMESH_CONTOURS,
                                  mp_drawing.DrawingSpec(color=(80, 110, 10), thickness=1, circle_radius=1),
                                  mp_drawing.DrawingSpec(color=(80, 256, 121), thickness=1, circle_radius=1)
                                  )

        # Draw right hand landmarks
        mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(80, 22, 10), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(80, 44, 121), thickness=2, circle_radius=2)
                                  )

        # Draw left hand landmarks
        mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(121, 22, 76), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(121, 44, 250), thickness=2, circle_radius=2)
                                  )

        # Draw pose detections
        mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(245, 117, 66), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(245, 66, 230), thickness=2, circle_radius=2)
                                  )

        # Display rep count on the image
        cv2.putText(image, 'Reps: {}'.format(counter), (10, 40),
        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

        cv2.imshow('Raw Webcam Feed', image)  # Rendering our results to the screen

        if cv2.waitKey(10) & 0xFF == ord('q'):  # How we close our webcam
            break

cap.release()
cv2.destroyAllWindows()
```

```python
import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe Holistic and Drawing utilities
mp_holistic = mp.solutions.holistic
mp_drawing = mp.solutions.drawing_utils
mp_face_mesh = mp.solutions.face_mesh

cap = cv2.VideoCapture(0)

# Function to calculate angle between three points
def calculate_angle(a, b, c):
    a = np.array(a)  # First point
    b = np.array(b)  # Mid point
    c = np.array(c)  # End point

    radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(a[1] - b[1], a[0] - b[0])
    angle = np.abs(radians * 180.0 / np.pi)

    if angle > 180.0:
        angle = 360 - angle

    return angle

# Variables to keep track of rep count
counter = 0
stage = None

# Initiate holistic model
with mp_holistic.Holistic(min_detection_confidence=0.5, min_tracking_confidence=0.5) as holistic:
    while cap.isOpened():  # Check if webcam is open or not
        ret, frame = cap.read()  # Our image in the frame

        if not ret:
            break

        # Recolor Feed
        image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        image.setflags(write=0)  # Set writable to False

        # Make Detections
        results = holistic.process(image)  # Update results with new detections

        # Recolor image back to BGR for rendering
        image.setflags(write=1)  # Set writable to True
        image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)  # Recoloring back to RGB to BGR

        # Extract landmarks for the right arm
        if results.pose_landmarks:
            landmarks = results.pose_landmarks.landmark

            # Get coordinates
            shoulder = [landmarks[mp_holistic.PoseLandmark.RIGHT_SHOULDER.value].x,
                        landmarks[mp_holistic.PoseLandmark.RIGHT_SHOULDER.value].y]
            elbow = [landmarks[mp_holistic.PoseLandmark.RIGHT_ELBOW.value].x,
                     landmarks[mp_holistic.PoseLandmark.RIGHT_ELBOW.value].y]
            wrist = [landmarks[mp_holistic.PoseLandmark.RIGHT_WRIST.value].x,
                     landmarks[mp_holistic.PoseLandmark.RIGHT_WRIST.value].y]

            # Calculate angle
            angle = calculate_angle(shoulder, elbow, wrist)

            # Visualize angle
            cv2.putText(image, str(angle),
                        tuple(np.multiply(elbow, [640, 480]).astype(int)),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2, cv2.LINE_AA
                        )

            # Curl counter logic
            if angle > 160:
                stage = "down"
            if angle < 30 and stage == 'down':
                stage = "up"
                counter += 1
                print(counter)

            # Calculate neck and torso inclination
            neck_angle = calculate_angle(
                [landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].y],
                [landmarks[mp_holistic.PoseLandmark.NOSE.value].x,
                 landmarks[mp_holistic.PoseLandmark.NOSE.value].y],
                [landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].y]
            )

            torso_angle = calculate_angle(
                [landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_HIP.value].y],
                [landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_SHOULDER.value].y],
                [landmarks[mp_holistic.PoseLandmark.LEFT_ANKLE.value].x,
                 landmarks[mp_holistic.PoseLandmark.LEFT_ANKLE.value].y]
            )

            cv2.putText(image, f'Neck Angle: {int(neck_angle)}', (10, 70),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)
            cv2.putText(image, f'Torso Angle: {int(torso_angle)}', (10, 110),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

            # Check if the person is bending below the threshold angle
            threshold_angle = 30  # Example threshold
            if neck_angle > threshold_angle:
                cv2.putText(image, 'Neck Bend Detected', (10, 150),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
            if torso_angle > threshold_angle:
                cv2.putText(image, 'Torso Bend Detected', (10, 190),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)

        # Draw face landmarks
        mp_drawing.draw_landmarks(image, results.face_landmarks, mp_face_mesh.FACEMESH_CONTOURS,
                                  mp_drawing.DrawingSpec(color=(80, 110, 10), thickness=1, circle_radius=1),
                                  mp_drawing.DrawingSpec(color=(80, 256, 121), thickness=1, circle_radius=1)
                                  )

        # Draw right hand landmarks
        mp_drawing.draw_landmarks(image, results.right_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(80, 22, 10), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(80, 44, 121), thickness=2, circle_radius=2)
                                  )

        # Draw left hand landmarks
        mp_drawing.draw_landmarks(image, results.left_hand_landmarks, mp_holistic.HAND_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(121, 22, 76), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(121, 44, 250), thickness=2, circle_radius=2)
                                  )

        # Draw pose detections
        mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS,
                                  mp_drawing.DrawingSpec(color=(245, 117, 66), thickness=2, circle_radius=4),
                                  mp_drawing.DrawingSpec(color=(245, 66, 230), thickness=2, circle_radius=2)
                                  )

        # Display rep count on the image
        cv2.putText(image, 'Reps: {}'.format(counter), (10, 40),
        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA)

        cv2.imshow('Raw Webcam Feed', image)  # Rendering our results to the screen

        if cv2.waitKey(10) & 0xFF == ord('q'):  # How we close our webcam
            break

cap.release()
cv2.destroyAllWindows()
```

RESULTS:


jupyter Untitled Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help
Trusted
Angle Plot
Angle Plot
Raw Webcam Feed
Reps: 18
Lower your arm a bit.
Keep your arm straight to avoid strain.


Raw Webcam Feed
Reps: 11
Lower your arm a
Keep your arm straight to avoid strain.
161.3691863637
Trusted
JupyterLab
Python 3 (ipykernel)


Raw Webcam Feed
Reps: 11
Lower your arm
Keep your arm straight to avoid strain.
178.22691006272052

FUTURE ADVANCEMENTS:

- **Advanced Posture Analysis Algorithms:** To implement more advanced posture analysis algorithms beyond basic angle measurement. This could include analysing posture dynamics, identifying asymmetries, and detecting subtle movement patterns associated with posture-related issues.

- **Integration of Biomechanical Data:** Integrate biomechanical data, such as joint forces and muscle activations, to provide a deeper understanding of the biomechanics underlying posture deviations. This data can enhance the accuracy of posture assessments and offer insights into muscular imbalances and potential injury risks.

- **Personalized Recommendations:** Develop algorithms that generate personalized posture correction recommendations based on individual biomechanics, habits, and specific postural challenges. Incorporate machine learning techniques to adapt and refine recommendations over time based on user feedback and progress.

- **Mobile Application and Wearable Integration:** Create a mobile application that complements the posture assessment system, allowing users to track their posture throughout the day using smartphone cameras or wearable

sensors. The app can provide continuous monitoring, reminders, and personalized feedback on posture habits.

- **Gamification and Engagement Strategies:** Implement gamification elements and engagement strategies to motivate users and promote consistent posture improvement. This could include challenges, rewards, progress tracking, and social sharing features to enhance user motivation and adherence to posture correction routines.

- **Long-Term Posture Monitoring:** Develop mechanisms for long-term posture monitoring and trend analysis to track changes in posture habits over weeks, months, or years. Use data analytics and visualization techniques to identify patterns, trends, and areas for improvement in posture behavior.

- **Integration with Health and Wellness Platforms:** Integrate the posture assessment system with existing health and wellness platforms, electronic health records (EHRs), and fitness tracking apps. This interoperability can facilitate holistic health management by incorporating posture data into overall wellness assessments and interventions.

- **AI-driven Coaching and Virtual Assistants:** Explore the use of AI-driven coaching and virtual assistants to provide real-time posture feedback, coaching tips, and interactive guidance. Develop natural language processing (NLP) capabilities for conversational interactions and personalized coaching dialogues.

- **Research Collaboration and Clinical Validation:** Collaborate with researchers, healthcare professionals, and ergonomic experts to conduct clinical validation studies and research trials. Validate the effectiveness of the posture assessment system in improving posture habits, reducing musculoskeletal risks, and enhancing overall well-being.

CONCLUSION:

In conclusion, this project report has explored the burgeoning field of AI-based tools for biomechanical analysis in rehabilitation. The integration of machine learning and deep learning algorithms has revolutionized the assessment and treatment of patients, offering unprecedented insights into movement patterns, gait, posture, and joint motion. These tools have not only proven to be accurate and reliable but also hold the potential to personalize rehabilitation interventions, optimize outcomes, and enhance the overall quality of care.

While challenges remain in terms of data availability, ethical considerations, and the need for interpretable AI models, the future of this field is undeniably promising. Continued research and development efforts in AI-driven biomechanical analysis tools are poised to transform rehabilitation medicine, empowering clinicians and improving the lives of patients worldwide. As technology continues to advance, we can anticipate even more sophisticated and impactful applications that will shape the future of rehabilitation.

REFERENCES:

- Bickley, S. J., & Smith, T. O. (2016). "The effectiveness of computer-based software on human posture: A systematic review." Physiotherapy Theory and Practice, 32(5), 429-442.
- Zhang, Z., et al. (2019). "Posture recognition based on deep learning for healthcare." Healthcare Technology Letters, 6(3), 72-76.
- Jiang, W., & Yin, Z. (2015). "Human activity recognition using wearable sensors by deep convolutional neural networks." Proceedings of the 23rd ACM international conference on Multimedia, 1307-1310.
- Choi, J., & Kim, D. (2020). Real-time posture monitoring system using machine learning and wearable sensors. Sensors (Basel, Switzerland), 20(11), 3090. [DOI: 10.3390/s20113090]
- Huang, M., Chen, Y., Lee, C., & Huang, H. (2019). A real-time posture monitoring system based on deep learning and computer vision. IEEE Access, 7, 150726-150736. [DOI: 10.1109/ACCESS.2019.2942893]
- Zhong, Y., Fang, X., Zhu, J., & Zhao, J. (2021). Mediapipe: A versatile framework for multimedia understanding. IEEE MultiMedia, 28(3), 6-14. [DOI: 10.1109/MMUL54769.2021.3062780]
- Kim, S., Kim, D., & Kim, J. (2018). A review of posture recognition techniques for real-time applications. IEEE Transactions on Systems, Man, and Cybernetics: Systems, 48(11), 1905-1918. [DOI: 10.1109/TSMC.2017.2762963] American Academy of Orthopaedic Surgeons. (2020). Proper sitting posture. Retrieved from https://orthoinfo.aaos.org/en/diseases--conditions/proper-sitting-posture
- Healthline Media. (2021). What is good posture? The ultimate guide. Retrieved from https://www.healthline.com/health/good-posture
- World Health Organization. (2019). Musculoskeletal conditions. Retrieved from https://www.who.int/news-room/fact-sheets/detail/musculoskeletal-conditions
- Fernandes, A. B., & Ferreira, J. P. (2021). Ergonomics in the workplace: A review of current practices and challenges. International Journal of Environmental Research and Public Health, 18(12), 6270. [DOI: 10.3390/ijerph18126270]